Guida Illustrata

Web Application Exploit SQLi

1. Sintesi

L’obiettivo di questa guida illustrata è spiegare ad un utente medio come eseguire un exploit SQLi su una Web Application.

---

2. Introduzione

Un exploit SQL Injection (SQLi) è un attacco da parte di un malintenzionato al database di una Web Application. L'attaccante scrive dei **pezzi di codice** che vanno a modificare la struttura logica della query originale che il programmatore aveva previsto.

3. Configurazione iniziale

La precauzione che si raccomanda è di svolgere l'attività in un ambiente controllato, utilizzando una macchina virtuale **Kali Linux** su un target **Metasploitable 2**.

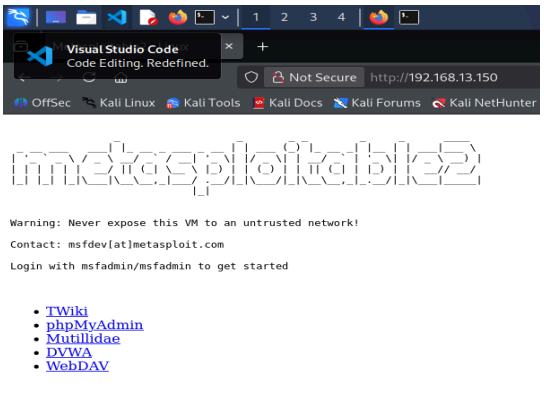
Il primo passo è configurare gli indirizzi IP delle Macchine Virtuali

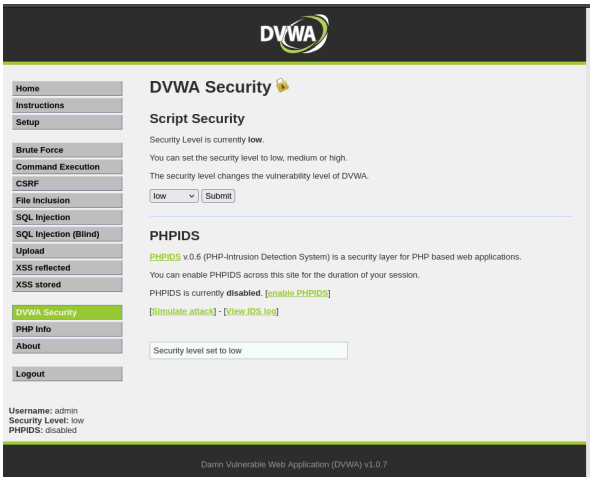
Kali Linux: 192.168.13.100/24

Metasploitable 2: 192.168.13.150/24

3.1 Analisi

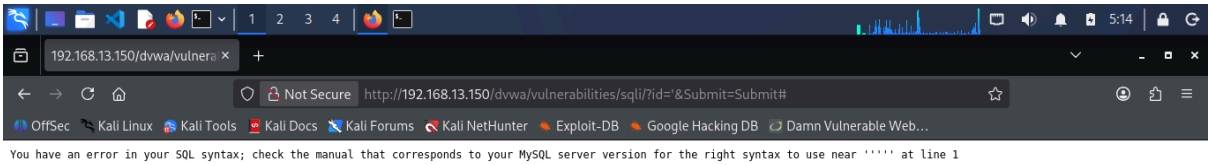
Il primo passo è accedere all'interfaccia di DVWA tramite l'indirizzo IP del target 192.168.13.150, impostare il livello di sicurezza su **LOW** ed accedere alla sezione **XSS Stored**.





3.2 Verifica della Vulnerabilità

Per testare se il campo "User ID" sia vulnerabile a SQL Injection, è necessario inserire un singolo apice ('). Se l'applicazione restituisce un errore di sintassi MySQL, conferma che l'input dell'utente non è correttamente filtrato e di conseguenza è possibile iniettare codice SQL.

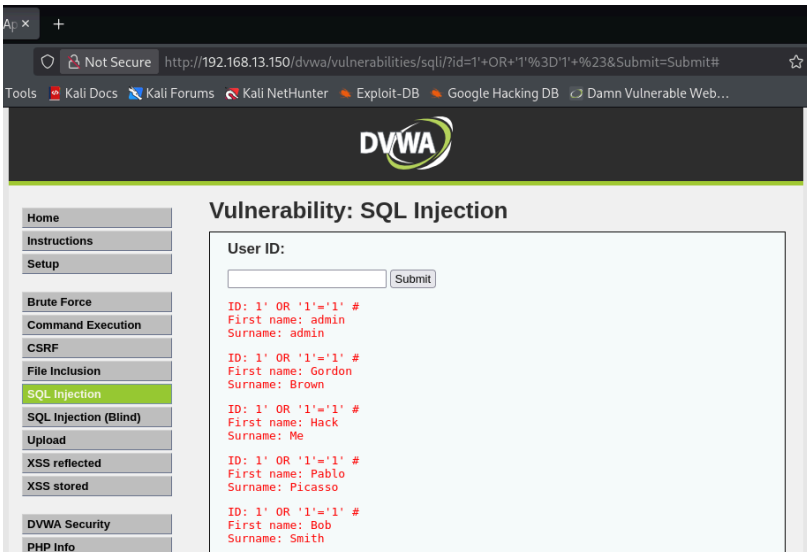


---

4. Enumerazione degli Utenti

Successivamente si passa all'estrazione di tutti i record presenti nella tabella degli utenti, utilizzando una condizione "sempre vera" tramite l'operatore `OR`.

- **Payload utilizzato:** `' OR '1'='1' #`
- **Spiegazione:** L'istruzione `OR '1'='1'` forza il database a restituire ogni riga della tabella, mentre il simbolo `#` commenta il resto della query originale impedendo errori del comando che mostra l'elenco completo degli utenti nel sistema (admin, Gordon, Pablo, ecc.).

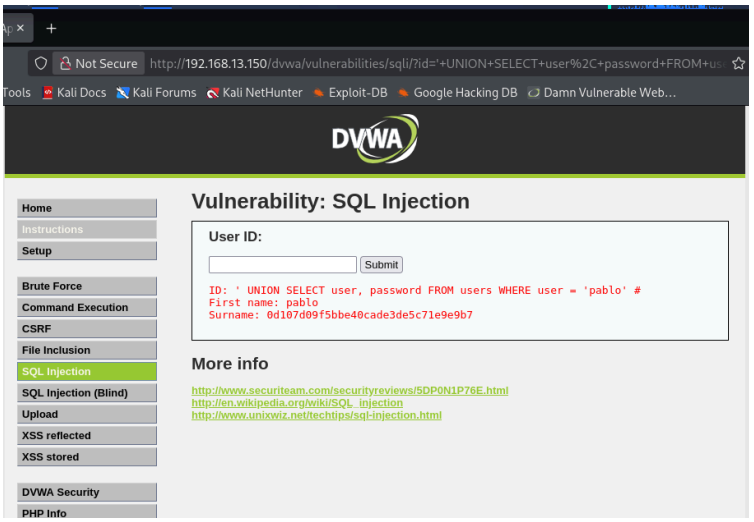


---

5. Estrazione dell'Hash (UNION-Based)

Una volta identificato l'utente target (**Pablo**), l'obiettivo è quello di recuperare l'hash della sua password, utilizzando la tecnica **`UNION SELECT`** per unire i risultati della query originale con i dati sensibili della tabella users.

- **Payload:** `' UNION SELECT user, password FROM users WHERE user = 'pablo' #`
- **Risultato:** Nome: pablo, Password (hash): 0d107d09f5bbe40cade3de5c71e9e9b7



---

6. Cracking della Password con John the Ripper

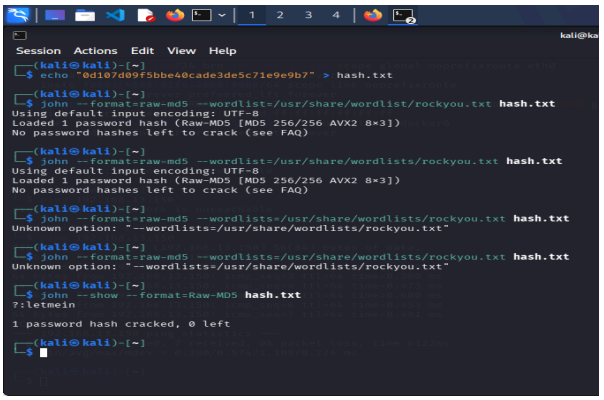
L'hash ottenuto (0d107d09f5bbe40cade3de5c71e9e9b7) è codificato in formato MD5, per recuperare la password in chiaro, si consiglia l'utilizzo del tool **John the Ripper** su Kali Linux, seguendo i seguenti passaggi:

1.Salvataggio dell'hash: L'hash deve essere salvato in un file, in questo caso chiamato hash.txt.

2.Esecuzione del programma: eseguire John The Ripper da terminale Kali con il comando:

john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt

3. Risultato: una volta eseguito il terminale di Kali Linux mostra il risultato dell'attacco brute-force e la password di Pablo è **letmein**.



---

7. Conclusione

A questo punto l'attacco è andato a buon fine. Grazie alla vulnerabilità SQL Injection, è stato possibile bypassare i controlli di autenticazione, estrarre il database degli utenti e decifrare la password amministrativa.

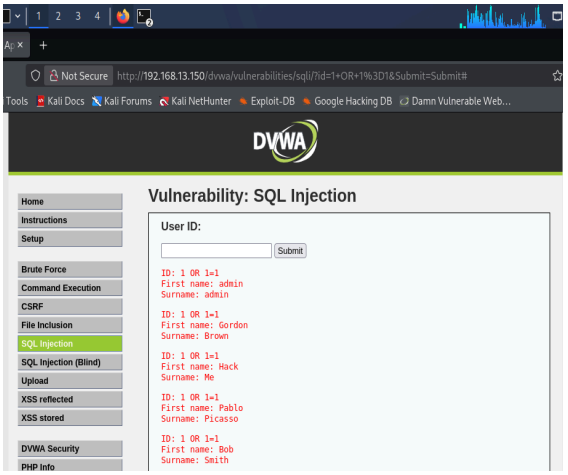
- **User:** pablo
- **Password in chiaro:** letmein

8. Livello Medium

E’ possibile anche replicare il procedimento impostando la DVWA ad un livello di sicurezza **Medium.** E recuperare informazioni vitali da altri database collegati.

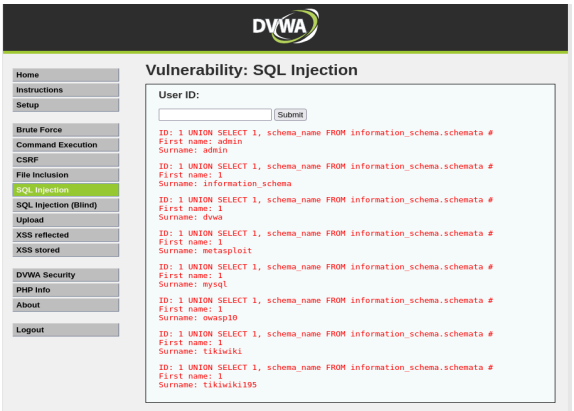
8.1 Procedimento

A questo livello di sicurezza il programma introduce una difesa specifica: l'**escaping dei caratteri**. Nel codice sorgente del livello Medium, l'input dell'utente viene passato attraverso una funzione che cerca i caratteri "pericolosi" (come l'apice **'**) e aggiunge un backslash davanti (**\'**). E’ consigliabile dunque inserire il numero 1 invece dell’apice (**‘**)



Per recuperare informazioni da altri database invece, è necessario modificare il payload. Anche qui è necessario sostituire gli apici con il numero.

- **Payload:** 1 UNION SELECT 1, schema_name FROM information_schema.schemata #
- **Risultato:** questo payload restituisce informazioni sui database collegati come metasploit, mysql, etc.



9. Considerazioni finali

L'analisi condotta attraverso i livelli **Low** e **Medium** di DVWA dimostra come una singola riga di codice non protetta possa mettere a nudo l'intero cuore di una Web Application: il suo database.